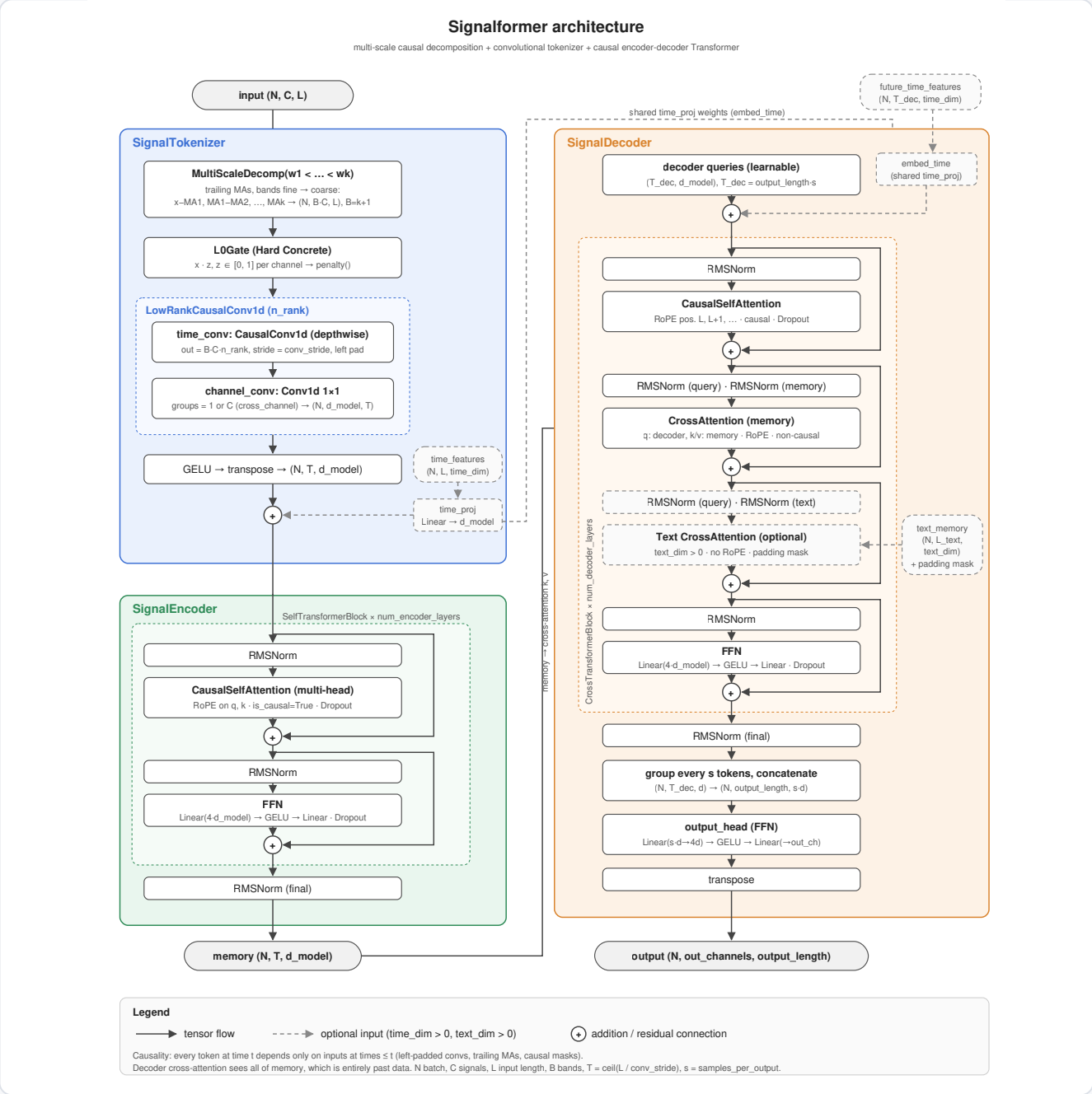
Signalformer

A causal encoder-decoder Transformer for multivariate signals. Signals are split into multi-scale bands, tokenized by a low-rank causal convolution, encoded with causal self-attention, and decoded by learnable future queries into an output of configurable width and frequency.



Solid arrows carry tensors. Dashed elements are optional and exist only when `time_dim > 0` or `text_dim > 0`.

Overview

The model has three submodules, run in order. Shapes use `N` for batch, `C` for input signals, `L` for input length, `B` for bands per signal, `T = ceil(L / conv_stride)` for tokens and `s` for `samples_per_output`.

MODULE	INPUT	OUTPUT	ROLE
SignalTokenizer	(N, C, L)	(N, T, d_model)	One token per `conv_stride` input steps
SignalEncoder	(N, T, d_model)	(N, T, d_model)	Context over the input window (memory)
SignalDecoder	(N, T, d_model)	(N, out_channels, output_length)	Forecast of all future steps at once

```
model = Signalformer(in_channels=5, in_len=128, d_model=64, kernel_size=3,
                     output_length=4, num_heads=4, ma_windows=(5, 20, 60))
y = model(x)                                    # x: (N, 5, 128) -> y: (N, 1, 4)
loss = task_loss(y, target) + lam * model.penalty()
```

SignalTokenizer

`MultiScaleDecomp` → `L0Gate` → `LowRankCausalConv1d` → `GELU` → `+ time features`

MultiScaleDecomp `(N, C, L) → (N, B·C, L)`

For strictly increasing windows `w1 < … < wk`, each signal is split into `k + 1` bands with trailing moving averages, from fine to coarse:

```
x − MA_w1,  MA_w1 − MA_w2,  …,  MA_w(k−1) − MA_wk,  MA_wk
```

The bands sum back to the input exactly. Each band holds variation at one time scale, so the gate can keep or drop scales independently, and small fine-scale bands are not dominated by the magnitude of the coarse band.

- Moving averages pad on the left with the first value, so each band at time *t* uses only inputs at times ≤ *t*.
- The first `max(windows) − 1` steps use incomplete windows. Drop them as warm-up during training, and keep `in_len` well above the largest window.
- The raw signal is not fed alongside the bands. It equals their sum, so the inputs would be perfectly collinear.
- Channels are ordered by band: band 0 of every signal, then band 1 of every signal, and so on. Gate index `i` maps to band `i // C` of signal `i % C`.

> This is wavelet-like, not a wavelet transform. Filters are nested box filters rather than scaled copies of a mother wavelet, bands are not orthogonal, box filters leak across frequencies, there is no downsampling, scales are arbitrary rather than dyadic, and detail bands have only one vanishing moment. It is closest to an undecimated Laplacian pyramid built from moving averages.

L0Gate `(N, B·C, L) → (N, B·C, L)`

A Hard Concrete gate (Louizos et al., 2018) learns one weight per channel that can be exactly zero. During training the gate samples a stretched, clipped sigmoid; at evaluation it is deterministic. `penalty()` returns the expected number of open channels, a differentiable approximation of the L0 norm.

- Standardize inputs first, or gate values reflect feature scale rather than importance.
- Set `lam` so the penalty is roughly 1–10% of the task loss to start. Too large closes every gate.
- Give `log_alpha` its own optimizer group with a higher learning rate and `weight_decay=0`.
- Correlated channels may be selected arbitrarily. Trust a selection only if it is stable across seeds and validation folds.

LowRankCausalConv1d `(N, B·C, L) → (N, d_model, T)`

A depthwise separable causal convolution, equal to the sum of `n_rank` rank-1 convolutions:

```
W[o, c, j] = Σ_r  B[o, c, r] · a[c, r, j]
```

- `time_conv` is a depthwise `CausalConv1d` (`groups = B·C`). Each channel is filtered along time on its own, with `n_rank` kernels per channel and `(kernel_size − 1)·dilation` zeros padded on the left.
- `channel_conv` is a 1×1 convolution. At each step it mixes channels, which is the same as a per-step `Linear`, so it is causal by construction.
- No activation sits between the two convolutions, otherwise the low-rank factorization no longer holds. The rank is capped at `min(kernel_size, d_model)`, so `n_rank ≥ kernel_size` is as expressive as a full `Conv1d`.
- Only zero padding is safe. Reflect or circular padding would copy future values into the left pad.

A GELU follows the convolution, then the tensor is transposed to `(N, T, d_model)` tokens.

Patching with `conv_stride` `L → T = ceil(L / conv_stride)`

With `conv_stride = S > 1`, `time_conv` emits one token per `S` input steps, so each token summarizes a patch. This is the same operation as PatchTST's patch embedding: a patch of length `P` taken every `S` steps and projected linearly is a convolution with `kernel_size = P` and stride `S`. Set `kernel_size = conv_stride` for non-overlapping patches and `kernel_size > conv_stride` for overlapping ones (for example 16 and 8). A shorter token sequence also cuts the O(T²) cost of attention.

- The stride grid is aligned so the last token ends at the last input step. Token `j` ends at input step `(L − 1) % S + j·S`, so the most recent data is always used even when `L` is not a multiple of `S`.
- `kernel_size` must be at least `conv_stride`, otherwise input steps between kernels would never be seen. The model rejects such settings.
- Time features are taken from the last input step each token covers.
- RoPE positions stay in input-time units on the decoder side: memory token `j` sits at `(L − 1) % S + j·S` and forecast step `k` at `L + k`, so cross-attention still sees the real time gap. Encoder self-attention uses token indices, which scales every distance by the same factor.

> `n_rank` matters much more once patches are long. `time_conv` compresses each channel's patch into `n_rank` numbers before channels are mixed, so `kernel_size = 16` with `n_rank = 1` keeps only one weighted average per channel and loses the shape inside the patch. `n_rank ≥ kernel_size` makes the tokenizer as expressive as a full patch projection; values between a quarter and a half of `kernel_size` trade capacity for fewer parameters.

Signal independence with `cross_channel`

By default `channel_conv` mixes the bands of all signals into each token. With `cross_channel=False` it is grouped by signal: each signal mixes only its own bands and owns `d_model / C` dimensions of the token, so `d_model` must be divisible by `C`. Before the grouped convolution the channels are reordered from band-major to signal-major, so each signal's bands are contiguous.

Mixing signals early can overfit to correlations that do not persist, while keeping them apart can miss real interactions. Which works better depends on the data, so compare both settings. Signals stay independent only inside the tokenizer; attention layers still combine them. This is weaker than the full channel independence of PatchTST, which runs every signal through the whole Transformer separately.

Time features `(N, L, time_dim) → + (N, L, d_model)`

Optional calendar features bypass the convolution. `time_proj` (a `Linear`) maps them to `d_model` and adds them to the tokens. The decoder reuses the same projection through `embed_time()`, so a given date maps to the same vector on both sides.

- Encode cyclical fields such as day of week, month and day of year as sin/cos pairs.
- Use 0/1 flags for known events such as month end, holidays or scheduled events.
- Avoid absolute year features. Test periods always lie outside the training range, so the model would extrapolate. Use features that describe the current state of the system instead, or a linear scaling such as `(year − base) / 10` if a year is unavoidable.

SignalEncoder

`SelfTransformerBlock × num_encoder_layers` → `RMSNorm` → `memory`

Each block is pre-norm with two residual sub-layers:

```
x = x + Dropout(CausalSelfAttention(RMSNorm(x)))
x = x + Dropout(FFN(RMSNorm(x)))          # Linear(4·d_model) → GELU → Linear
```

CausalSelfAttention

- Uses `F.scaled_dot_product_attention(..., is_causal=True)`, which picks a fused kernel such as FlashAttention when one is available.
- RoPE rotates q and k per head after the projections, so attention scores depend only on the distance between two positions. Applying it to embeddings before the projections would lose that property.
- Attention selects time steps, not features. Its weights are not a reliable measure of feature importance; use the gate values or permutation importance instead.

Normalization and dropout

- RMSNorm normalizes over features only, so it never mixes time steps and stays causal. BatchNorm along time would leak statistics from the future.
- RMSNorm does not subtract the mean. Remove offsets from the inputs before the model.
- Pre-norm leaves the last block's output unnormalized, so a final RMSNorm produces the memory.
- Dropout regularizes; normalization only stabilizes training. Start near 0.1 and raise it if the gap between training and validation loss is large.

SignalDecoder

`queries + future time` → `CrossTransformerBlock × num_decoder_layers` → `RMSNorm` → `group s tokens` → `output_head`

Learnable queries `(output_length·s, d_model)`

The decoder is non-autoregressive. It holds one learnable query per future time step, at the input frequency, and predicts every step in one pass. The queries also tell the decoder which step it is predicting. When `time_dim > 0`, the embedded `future_time_features` are added to the queries. Those features may only contain information known in advance.

CrossTransformerBlock

Pre-norm with up to four residual sub-layers:

```
y = y + Dropout(CausalSelfAttention(RMSNorm(y), offset=L))
y = y + Dropout(CrossAttention(RMSNorm(y), RMSNorm(memory)))
y = y + Dropout(TextCrossAttention(RMSNorm(y), RMSNorm(text)))   # only if text_dim > 0
y = y + Dropout(FFN(RMSNorm(y)))
```

- Self-attention is causal. Future step *k* sits at RoPE position `L + k`, so step *k* sees only steps ≤ *k*.
- Cross-attention to memory is not causal, since memory holds only past data. RoPE gives queries positions `L, L+1, …` and memory tokens the input step they end at, so scores reflect the real time gap at any `conv_stride`.
- Memory has its own RMSNorm inside each block.

Text cross-attention `(N, L_text, text_dim)`

An optional branch attends to embeddings from an external text encoder. It has no RoPE, because text tokens have no positions aligned with the signal.

- `text_padding_mask` marks padding with `True`, matching `nn.MultiheadAttention`. It is converted for `scaled_dot_product_attention`, where `True` means allowed.
- If every text token of a sample is padding, its attention output is set to zero after `out_proj`, so that sample behaves exactly as if no text were given. Passing `text_memory=None` skips the branch.
- Only use text that was available before the end of the input window, judged by when it became available rather than when the event it describes happened.
- A pretrained text encoder may have been trained on data from after the training period. Prefer encoders whose training cutoff precedes the evaluation period.

Aggregation and output head `(N, output_length·s, d_model) → (N, out_channels, output_length)`

After the final RMSNorm, every `s = samples_per_output` consecutive tokens are concatenated into one vector of size `s·d_model`. Concatenation keeps the order of tokens inside each period, which averaging would lose. A per-step FFN (`Linear → GELU → Linear`) maps each vector to `out_channels` values, and a final transpose returns the input layout.

The decoder itself always runs at the input frequency. Only this last step makes the output coarser, for example `s = 7` for daily input and weekly output. Training targets must be aggregated over the same periods.

Causality

For any time *t* in the input window, the token at *t* depends only on inputs at times ≤ *t*. Each component keeps this property:

COMPONENT	MECHANISM
`MultiScaleDecomp`	Trailing moving averages, left-padded with the first value
`time_conv`	Left zero-padding of `(kernel_size − 1)·dilation`; with a stride, every window ends at or before the token's last input step
`channel_conv`	Kernel size 1 touches only the current step
`RMSNorm, FFN`	Operate per time step over features
`Self-attention`	`is_causal=True` in encoder and decoder
`Cross-attention`	Memory and text contain only data from before the forecast time

> The model cannot protect against leakage in the data pipeline. Normalize inputs with statistics computed from past data only, validate with time-ordered (walk-forward) splits, and keep inputs in fp32 under mixed precision. bf16 autocast needs no gradient scaler; fp16 needs `torch.amp.GradScaler`.

Parameters

NAME	DEFAULT	MEANING
`in_channels`	—	Number of input signals `C`
`in_len`	—	Input length `L` (currently unused by the modules)
`d_model`	—	Token width; must be divisible by `num_heads`
`kernel_size`	—	Temporal kernel of `time_conv` (patch length when `conv_stride > 1`)
`output_length`	—	Number of output steps
`num_heads`	—	Attention heads; `d_model / num_heads` must be even for RoPE
`ma_windows`	`(5, 20, 60)`	Strictly increasing MA windows; `B = len + 1`
`n_rank`	`1`	Number of rank-1 terms in the tokenizer convolution; raise it with long patches
`num_encoder_layers`	`2`	Encoder blocks
`num_decoder_layers`	`2`	Decoder blocks
`dropout`	`0.1`	Dropout in attention and residual branches
`time_dim`	`0`	Width of time features; 0 disables them
`out_channels`	`1`	Values per output step
`samples_per_output`	`1`	Input steps aggregated into one output step (`s`)
`text_dim`	`0`	Width of text embeddings; 0 disables text cross-attention
`conv_stride`	`1`	Input steps per token (patching); must be ≤ `kernel_size`
`cross_channel`	`True`	Mix bands across signals in the tokenizer; `False` groups by signal and needs `d_model % C == 0`